Saravanan Natarajan
Solution Architect @Bosch

OpenCost in Kubernetes: SaaSification of AI services

Why SaaSification ?

01



Organizations — Now AI-Powered

02



3 Problems & AI Solutions

03



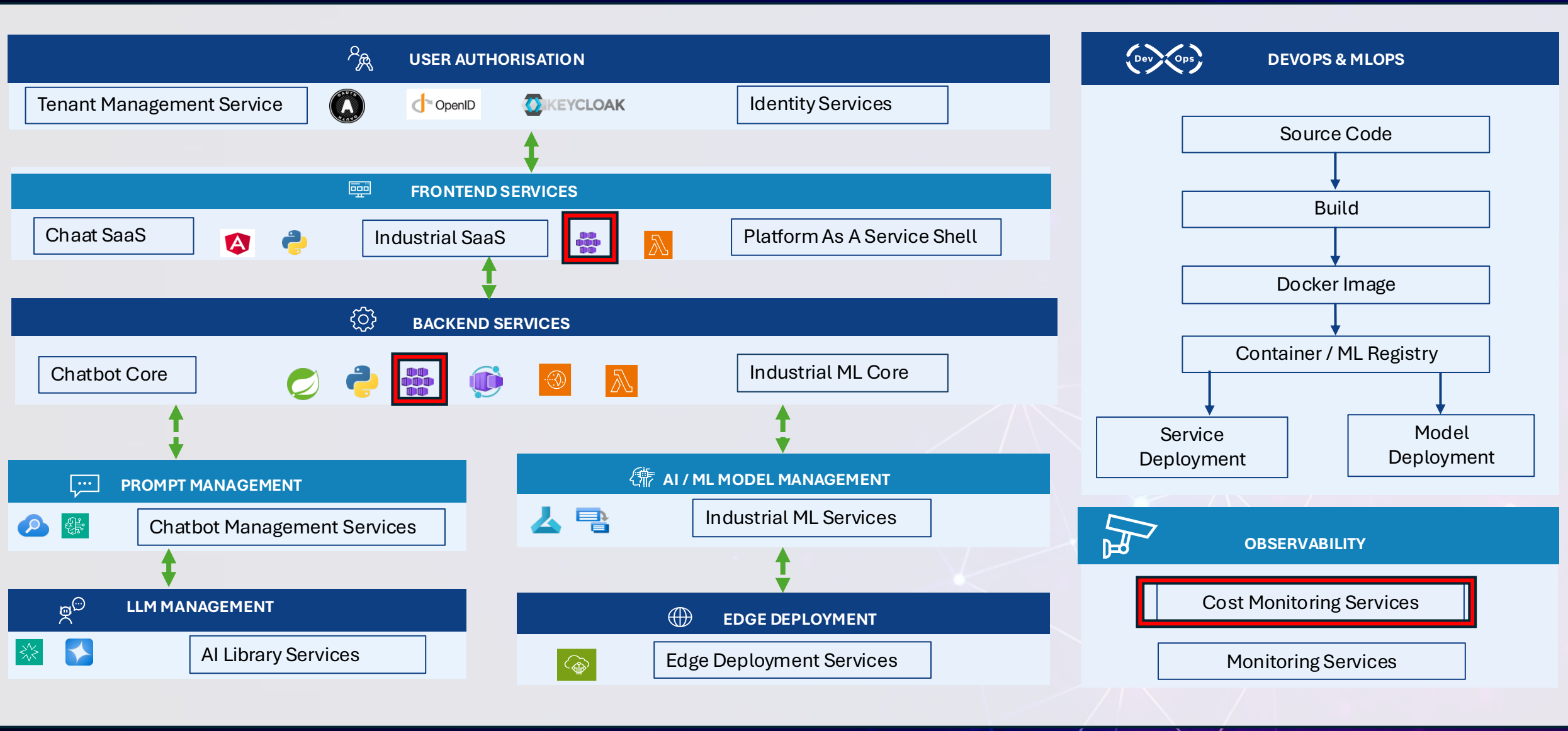
Technology + Finance + Business

04



Cost Observability on Kubernetes

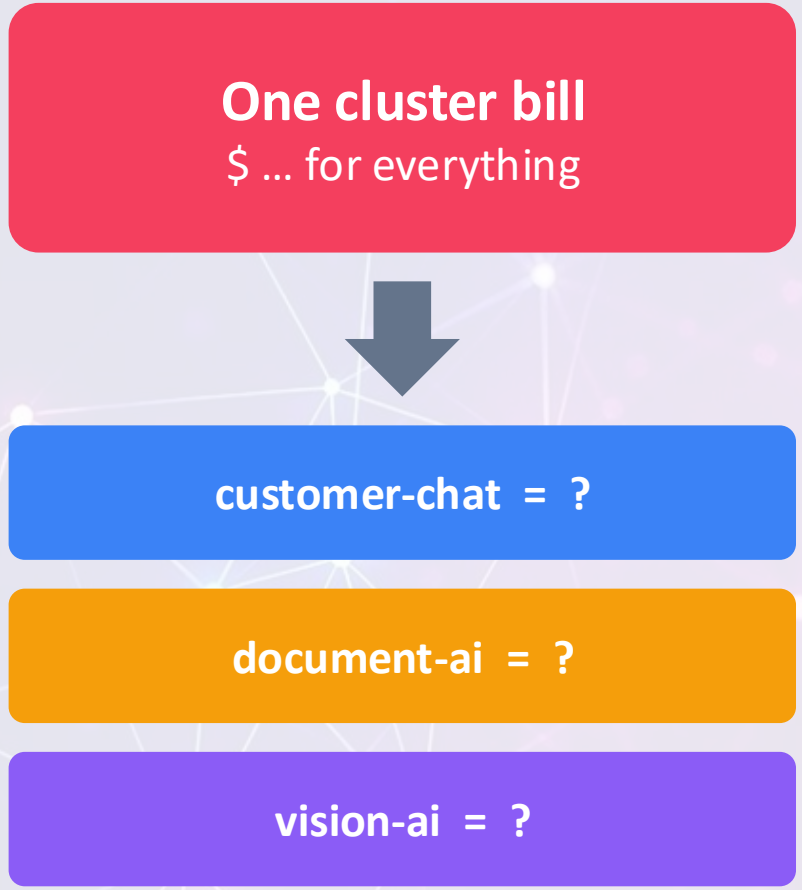
High-level Architecture: *SaaSification of AI Services - Deployment View*



FinOps – Financial Operations

Who is paying for our AI services?

- Multiple AI products share one Kubernetes cluster
- Each product has a very different CPU and memory profile
- The cloud bill arrives per cluster, not per product or team
- Usage metrics (requests, tokens) show demand but not infrastructure cost
- Finance, product, and engineering lack a shared view of cost ownership



Three open-source building blocks

OpenCost

CNCF project for Kubernetes cost allocation.

Combines CPU and memory requests with node pricing to estimate cost per namespace, pod, and label.

Prometheus

Collects and stores time-series metrics.

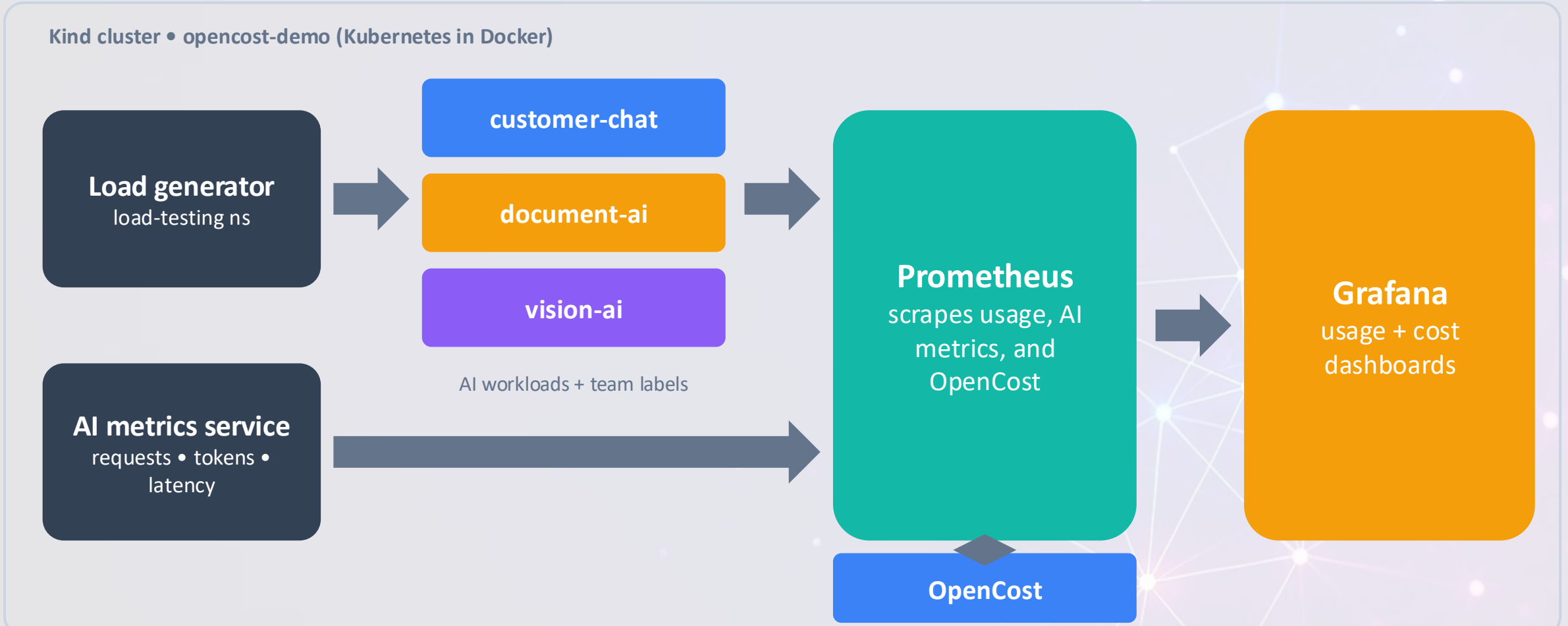
Scrapes Kubernetes resource usage, OpenCost cost metrics, and AI request, token, and latency metrics.

Grafana

Visualises everything stored in Prometheus.

Dashboards compare AI service usage with OpenCost allocation and estimated monthly cost.

How the pieces fit together



THE DEMO WORKLOADS

Three AI services, three owning teams

customer-chat

Team: conversational-ai

3 replicas (HPA 3–8)
500m CPU • 1Gi memory per pod

Latency-sensitive, CPU-light chat API

document-ai

Team: document-intelligence

2 replicas (HPA 2–6)
1 CPU • 2Gi memory per pod

Memory-heavy document processing workers

vision-ai

Team: computer-vision

2 replicas (HPA 2–5)
1.5 CPU • 3Gi memory per pod

CPU-intensive image inference API

Labels on every resource: team • opencost.io/team • app.kubernetes.io/part-of=opencost-ai-demo

SIZING

Resource requests drive allocated cost

Service	Replicas	CPU / pod	Memory / pod	Total CPU	Total memory
customer-chat	3	500m	1Gi	1.5 cores	3Gi
document-ai	2	1	2Gi	2.0 cores	4Gi
vision-ai	2	1500m	3Gi	3.0 cores	6Gi
Total (baseline)	**7**			**6.5 cores**	**13Gi**

Share of requested CPU



How OpenCost turns allocation into dollars

```
hourly_cost =
    container_cpu_allocation          × node_cpu_hourly_cost
  + container_memory_allocation_bytes × node_ram_hourly_cost / 1 GiB

monthly_cost = sum by (namespace, pod) (hourly_cost) × 730
```

Inputs

CPU and memory requests per container
Hourly CPU and RAM price per node

Grouping

PromQL: namespace, pod (1:1 with team)
OpenCost API/UI: team or opencost.io/team

Caveat

Allocation = requests, not usage
Kind uses OpenCost default pricing

WALKTHROUGH

Live demo

1 **Show the workloads**

```
kubectl get pods -A -l app.kubernetes.io/part-of=opencost-ai-demo
```

2 **Generate AI traffic**

```
kubectl -n ai-observability port-forward svc/ai-metrics-service 8080:8080
curl -X POST http://localhost:8080/infer -d '{...}'
```

3 **Open Grafana**

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

4 **Open OpenCost UI**

```
kubectl -n opencost port-forward svc/opencost 9090:9090
```

METRIC LINEAGE

Where container_memory_allocation_bytes comes from

1. K8s Manifest
resources.requests
memory: 1Gi / 2Gi / 3Gi

2. OpenCost (:9003)
Watches K8s Pod specs
Exports allocation gauge

3. Prometheus
ServiceMonitor scrape
Stores time series

4. Grafana
Usage vs. Request panel
× node_ram_hourly_cost

OpenCost

opencost-demo.yaml Ln 76–82

```yaml
resources:
  requests:
    cpu: 500m
    memory: 1Gi
  limits:
    cpu: "1"
    memory: 2Gi
```

Prometheus

opencost-values.yaml Ln 1–11

```yaml
opencost:
  prometheus:
    internal:
      serviceName: kube-prometheus-stack-prometheus
      namespaceName: monitoring
      port: 9090
  metrics:
    serviceMonitor:
      enabled: true
      additionalLabels:
        release: kube-prometheus-stack
```

Grafana

AI Service Usage (Request vs. Working Set):
sum by (namespace)
(container_memory_allocation_bytes)

AI Service Cost Allocation (GiB × Hourly Node RAM Price):
container_memory_allocation_bytes
* on (node) group_left node_ram_hourly_cost
/ (1024 * 1024 * 1024)

Key takeaways

Labels create ownership

Namespaces and team labels are what make cost attribution possible.

Requests are the cost contract

OpenCost allocates by requested CPU and memory, so right-size them.

Usage + cost = unit economics

Tokens and requests next to cost give cost per request or per 1K tokens.

Fully open source

OpenCost, Prometheus, Grafana, and Kind. Reproducible on a laptop.



Saravanan Natarajan

Manager II, Solution Architecture at Bosch Digital